COSC204 Computer Systems

Lecture 20: Synchronisation and Deadlock

Zhiyi Huang

School of Computing, University of Otago

Overview

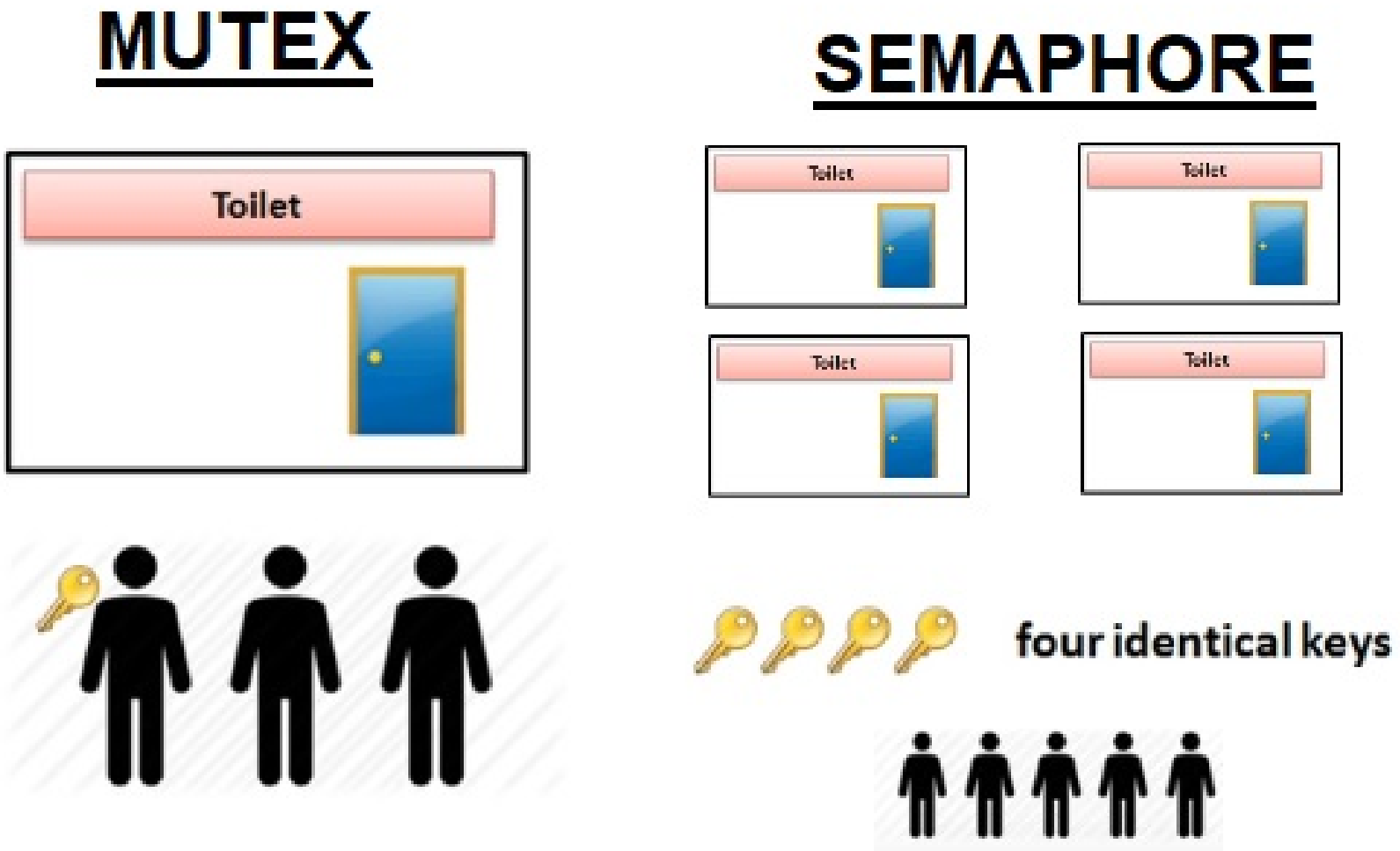
Last lecture:

- CPU scheduling algorithms
- Criteria for evaluating CPU scheduling algorithms

This lecture: synchronisation and deadlock

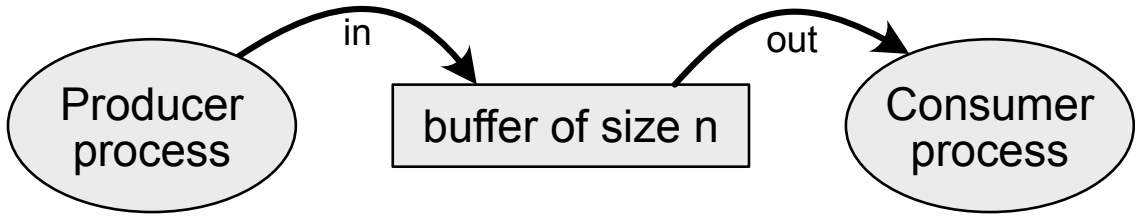
- Data race problems
- Critical sections
- Locking and hardware support
- Semaphores and mutex
- Deadlock

Lock (mutex) and semaphore



The bounded buffer problem

The producer-consumer pattern, using a bounded buffer, is a classic synchronization problem.



Private variables:

- `in`: the next position in the buffer to be written to (initially 0).
- `out`: the next position in the buffer to be read (initially 0).

Shared variables:

- `buffer`: the buffer itself.
- `counter`: the number of items in the buffer (initially 0).

Solution using a shared array

```
PRODUCER: repeat
            ...
            produce an item in nextp
            ...
            while counter = n do no-op;
            buffer[in] := nextp;
            in := in+1 mod n;
            counter := counter+1;
          until false;
CONSUMER: repeat
            while counter = 0 do no-op;
            nextc := buffer[out];
            out := out+1 mod n;
            counter := counter-1;
            ...
            consume the item in nextc
            ...
          until false;
```

An example of a problematic data race

Imagine the producer and consumer processes are executing in round-robin scheduling with a small time quantum.
Begin with `counter = 5` for both processes.

Now *interleave* the instructions for changing the value of `counter`:

```
Producer:                Consumer:
-------------------------------------
reg1 := counter;
reg1 := reg1 + 1;
                         reg2 := counter;
                         reg2 := reg2 - 1;
counter := reg1;
                         counter := reg2;
```

Critical sections help solve data races

A data race involves multiple processes/threads accessing the same data/memory location and at least one of them making modifications to the data.

To prevent data races, we must guarantee that certain sections of the cooperating programs are not interleaved.

These special protected sections are termed critical sections.

- Critical sections are usually protected by locking mechanisms.

Using locking to ensure mutual exclusion

Only one process can be executing its critical section at a time.

```
repeat
        acquire_lock(lk)
              critical section
        release_lock(lk)
until false;
```

Our counter example, now using locking

Begin with `counter = 5` for both processes.

```
Producer:                Consumer:
-----------------------------------
acquire_lock(l1);
reg1 := counter;
reg1 := reg1 + 1;        acquire_lock(l1);// wait until released
counter := reg1;
release_lock(l1)
                         reg2 := counter;
                         reg2 := reg2 - 1;
                         counter := reg2;
                         release_lock(l1);
```

Broken producer and consumer, with locking added

```
shared int l1;   initlock(&l1);
PRODUCER: repeat
            produce an item in nextp
            while counter = n do no-op;
            buffer[in] := nextp;
            in := in+1 mod n;
            acquire_lock(&l1);
            counter := counter+1;
            release_lock(&l1);
          until false;
CONSUMER: repeat
            while counter = 0 do no-op;
            nextc := buffer[out];
            out := out+1 mod n;
            acquire_lock(&l1);
            counter := counter-1;
            release_lock(&l1);
            consume the item in nextc
          until false;
```

Semaphores

Semaphores are a commonly used synchronisation tool making use of atomic operations.

A semaphore S is an integer variable that can only be accessed via two functions or system calls:

- `wait(S):`

  ```
  while S <= 0 do no-op;
  S := S - 1;
  ```

- `signal(S):`

  ```
  S := S + 1;
  ```

Producer/consumer using semaphores

- `empty_spaces`: semaphore initialised to the size of the buffer
- `full_spaces`: semaphore initialised to 0
- Each successful call to `wait` acquires unique lease on a resource (and `signal` releases it).

Producer:

```
repeat
    produce an item in nextp;
    wait(empty_spaces);
    wait(mutex);
    add nextp to buffer;
    signal(mutex);
    signal(full_spaces);
until(false);
```

Consumer:

```
repeat
    wait(full_spaces);
    wait(mutex);
    remove an item from buffer
      to nextc;
    signal(mutex);
    signal(empty_spaces);
    consume the item in nextc;
until(false);
```

Implementing locking uses hardware support

We need hardware support for locking!
One hardware solution is to disallow interrupts while a shared variable is being modified... But this only works for one CPU!

Modern computers provide special atomic instructions that will be executed atomically without interruption. Examples are CompareAndSwap (CAS) and testAndSet. Their atomicity is guaranteed by locking the memory bus.

`CAS(o, a, n)` definition: If the value at address `a` is `o`, write the value `n` to address `a` and then return true; Otherwise, return false, where `o` and `n` are integers, and `a` is the address of a memory location.

CAS-based implementation of a lock

```
void initlock(int *lk) {
  *lk = 0;
}

void acquire_lock(int *lk){
  int ov;
again:
  ov = *lk;
  if (ov != 0) goto again;
  if(CAS(ov, lk, 1) == false)  goto again;
}

void release_lock(int *lk){
*lk = 0;
}
```

Note the busy waiting. This is called a `spinlock()` in Linux.

CAS-based implementation of semaphores

- `wait(int *S):`

```
again:
    T = *S;
    if (T <= 0) goto again;
    if(CAS(T, S, T-1) == false) goto again;
```

- `signal(int *S):`

```
again:
    T = *S;
    if(CAS(T, S, T+1) == false) goto again;
```

Semaphores for mutual exclusion

The cooperating processes share a semaphore `mutex`, initialised to 1.

```
mutex = 1;
```

Process P_i:

```
repeat
        wait(&mutex);
                critical section
        signal(&mutex);
                remainder section
until false;
```

Queues can reduce the amount of busy waiting done

To reduce busy waiting, we can associate a *process queue* with each semaphore (and binary semaphores can be used as a mutex).

- `wait(S)`:

```
S.val := S.val - 1;
if S.val < 0
   then begin
        add this process to S.queue;
        block;
   end;
```

- `signal(S)`:

```
S.val := S.val + 1;
if S.val <= 0
   then begin
        remove a process P from S.queue;
        wakeup(P);
   end;
```

Interpreting values of semaphores with queues

Initial values: 1 for mutex, 0 for process synchronization, >1 allows for concurrent processes/threads.

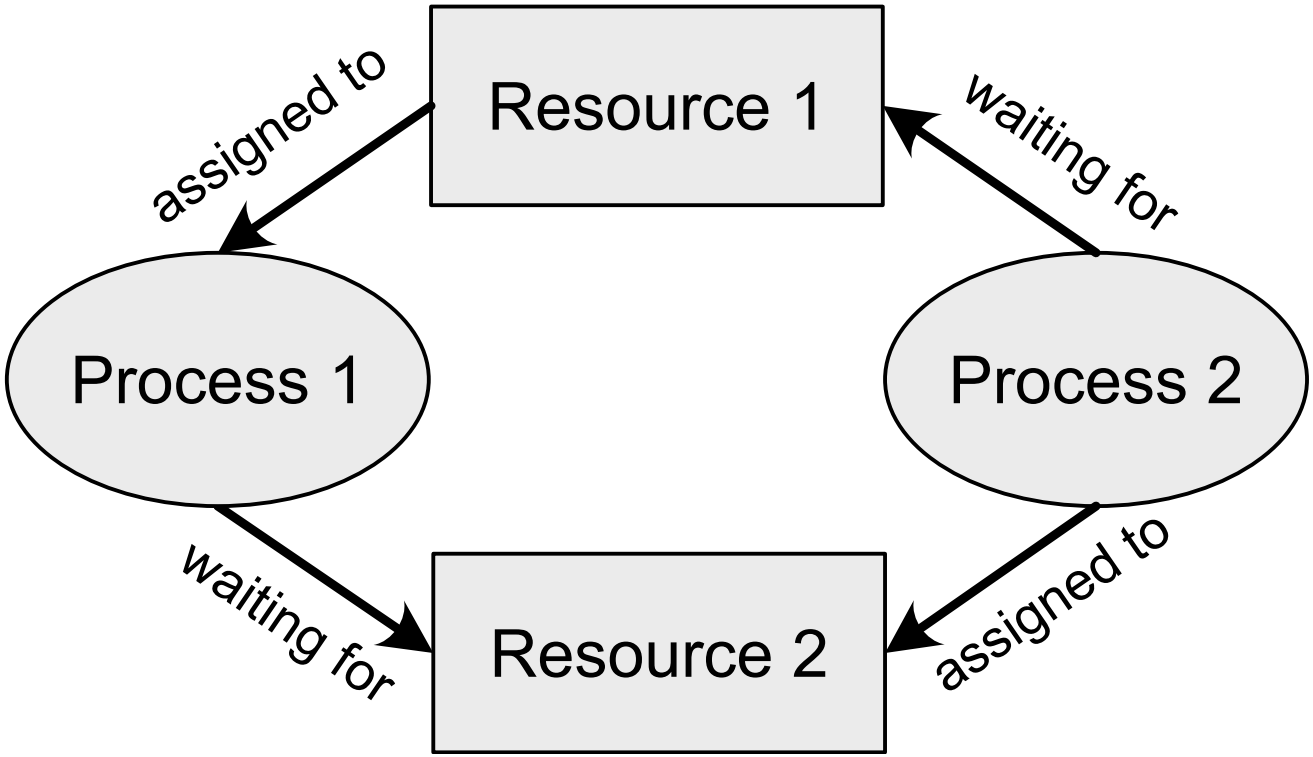
Semaphores that include queues can have values less than zero.

- Now if a process calls `wait` on a semaphore that has value 0 or lower, the semaphore is decremented anyway.
- Negative values tell us how many processes are waiting on that semaphore.

A semaphore can also be used to 'count' arbitrary positive values.

- If a process calls `signal` on a semaphore, it (atomically) increments that semaphore.
- Consecutive `signal`s can result in arbitrarily large positive values.

Deadlock: a situation we want to avoid



A typical deadlock example

```
      P1:                       P2:
-----------------------------------------
acquire_lock(l1);    acquire_lock(l2);
acquire_lock(l2);    acquire_lock(l1);

    ...                       ...
```

Definition of deadlock: a set of processes is in a deadlocked state if every process is waiting for an event that can only be caused by another process in the set.

Necessary conditions for deadlocks

1. Mutual exclusion: at least one of the held resources must be non-sharable.
2. Hold and wait: there must be at least one process that is holding a resource and waiting for another resource.
3. No preemption: a resource can only be released by the process that is holding it.
4. Circular wait:
   - P_1 is waiting on a resource held by P_2;
   - P_2 is waiting on a resource held by P_3;
   - ...
   - P_n is waiting on a resource held by P_1.

A common approach for avoiding deadlocks

Break the circular wait by acquiring the locks in the same order.

```
      P1:                          P2:
-------------------------------------------
acquire_lock(l1);    acquire_lock(l1);
acquire_lock(l2);    acquire_lock(l2);

    ...                          ...
```

Are there OS solutions for avoiding deadlock?

It would be useful for the operating system to be able to monitor the processes currently executing, and

- to detect if a deadlock has occurred; or
- to prevent deadlocks from ever occurring.

Readings

For this lecture, the relevant material is in Chapter 6 (Sections 1–7) of Silberschatz *et al*.